

Homework 4

Gradient Descent for Minimizing a 3-D Function

Project Overview

Gradient descent is a fundamental optimization technique widely used in machine learning and numerical analysis. In this project, you will implement the Gradient Descent Algorithm in Python to find the minimum of a two-variable function $f(x,y)$. You will:

- Define a differentiable function $f(x,y)$ with a known minimum.
- Implement gradient descent to iteratively approach the minimum.
- Experiment with different learning rates and initial points.
- Visualize the optimization path on 3D surface plots and 2D contour plots.

Learning Objectives

By completing this project, you will:

- Understand the mechanics of gradient descent in multivariate optimization.
- Learn how learning rate affects convergence.
- Explore the impact of initial points on convergence behavior.
- Develop skills in visualizing optimization trajectories.

Implementation Guide

Step 1: Define the Function and Its Derivative in Python

1. We choose a simple convex quadratic function with a clear minimum $(x,y)=(3, -2)$:
$$f(x,y)=(x-3)^2+(y+2)^2$$
2. Gradient:
$$\nabla f(x, y)=\left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right)=(2(x-3), 2(y+2))$$

Step 2: Implement the Gradient Descent Algorithm in Python

The update rule is:
$$x_{k+1}=x_k-\alpha \cdot \frac{\partial f}{\partial x}\left(x_k, y_k\right)$$

$$y_{k+1}=y_k-\alpha \cdot \frac{\partial f}{\partial y}\left(x_k, y_k\right)$$

Where:

- α = learning rate
- (x_k, y_k) = current point

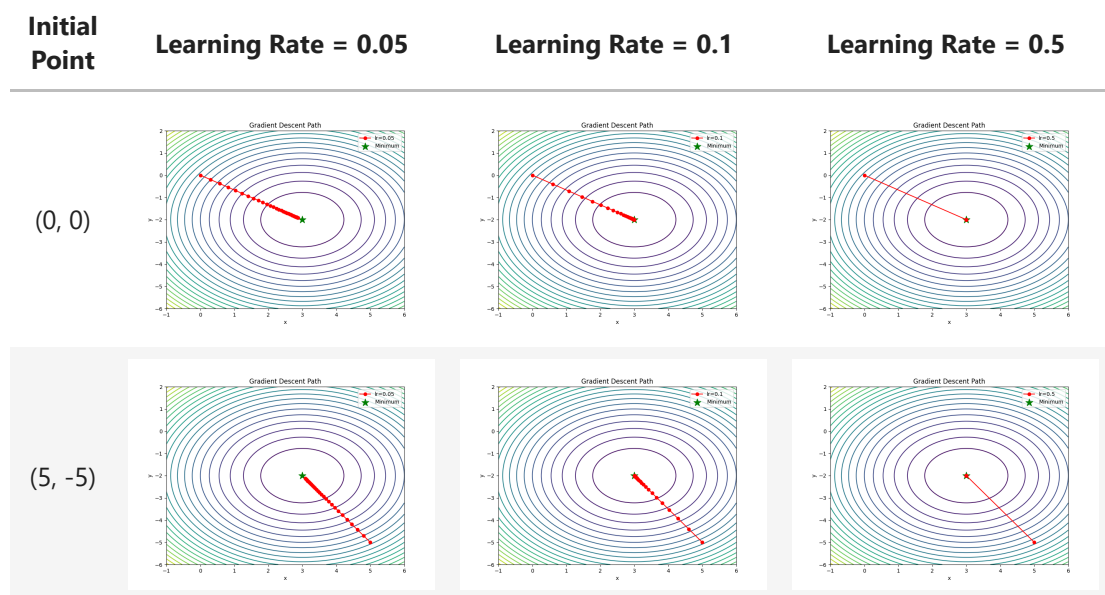
Step 3: Visualize the Optimization Path

Plots will clearly show how the initial point moves step by step toward the minimum, illustrating the effect of learning rate and initialization.

- Use these two initial points: (0,0), (5,-5)
- For each initial point, show the **contour** plot for different learning rates: 0.05, 0.1, 0.5

Sample result

- Case 1: Small learning rate ($\alpha=0.05$)
 - Converges slowly, requiring many iterations.
 - Path shows small steps toward the minimum.
- Case 2: Moderate learning rate ($\alpha=0.1$)
 - Converges efficiently to the minimum point.
 - Smooth trajectory.
- Case 3: Large learning rate ($\alpha=0.5$)
 - May overshoot or oscillate around the minimum.
 - Visualization shows zig-zagging behavior.



Deliverables

- Python code implementing gradient descent to find the minimum of a two-variable function.
- Plots showing convergence paths for different learning rates and initial points.
- Short report discussing:
 - Effect of learning rate on convergence.
 - Effect of initial point choice.
 - Observations about stability and efficiency.

